Explanation
COSC 3020
Lab Practice 12
Alexander Warren
2026-05-06

This program utilizes a dynamic bottom up approach to calculate the probability. Initially I wrote out the naive solution. The issue with this solution is that it is very inefficient, so we require a much more efficient solution achieved via dynamic programming. This involves keeping track of a 2D table, where each index stores the probability of P(i, j, n). We have base cases for when A or B has hit the target. From here, each entry for the table is computer from the top down and then the table entry at the desired location is returned. The payout function splits the pot based on the probability of whoever wins.

This solution is both correct and efficient. Each entry is calculated before it is needed and it avoids repeated calculations by storing the previously calculated entries, which is extremely efficient compared to the naive solution. That being said, our time and space complexity is still O(n^2). This solution’s correctness is proved by the test cases, visible in main() in the solution.py file.